



TECHNISCHE
UNIVERSITÄT
WIEN

ottobock.

Framework for System Simulation

Thesis – System Modeling

Florian Frech, 15.09.2025

1. Framework Architecture Overview

2. Demo Model: Controlled Pendulum

- Suitability of the Demo Model
- Block Diagram
- Component Description

3. Equation-based Modeling

- Reasons for using OpenModelica and OMEdit
- Subsystem Modeling (Examples: Pendulum, Reference, Angle Encoder)
- Building and Simulation of a System Model in OMEdit
- OMPython: Using Modelica Models in Python

4. Co-Simulation with FMUs using FMPy

- FMI / FMU Recap
- Workflow: From Modelica to FMU to Python
- Co-Simulation with FMUs using OMPython (Demo Model)

5. Heterogeneous Simulation Workflow:

- OpenSim Pendulum with Torque Actuation
- Unified Co-Simulation Approach
- Qualitative Simulation Result Comparison (Modelica / FMUs / FMUs & OpenSim)

6. Demo Model Extension: Pendulum with Wall Impact

- Motivation for Wall Impact Modeling
- Modelica: Continuous vs. Discrete Wall Impact
- OpenSim: Hunt-Crossley Force Contact
- FEM: Static and Dynamic Impact (PreProMax + CalculiX)

7. Requirements Coverage

- Modeling
- Simulation
- Heterogeneity
- User Interaction

8. Next Steps



User Interface Layer:

- Uses OMEdit for equation-based model definition (Modelica)
- Uses OpenSim GUI for visualization of OpenSim Model
- Configuration files:
 - File paths to FMUs, OpenSim models, FEM models
 - Defining values for parameters and initial values
 - Simulation settings: start/stop time, step size, solver settings
 - Logging and output options
- Jupyter Notebook: used as interactive orchestration environment

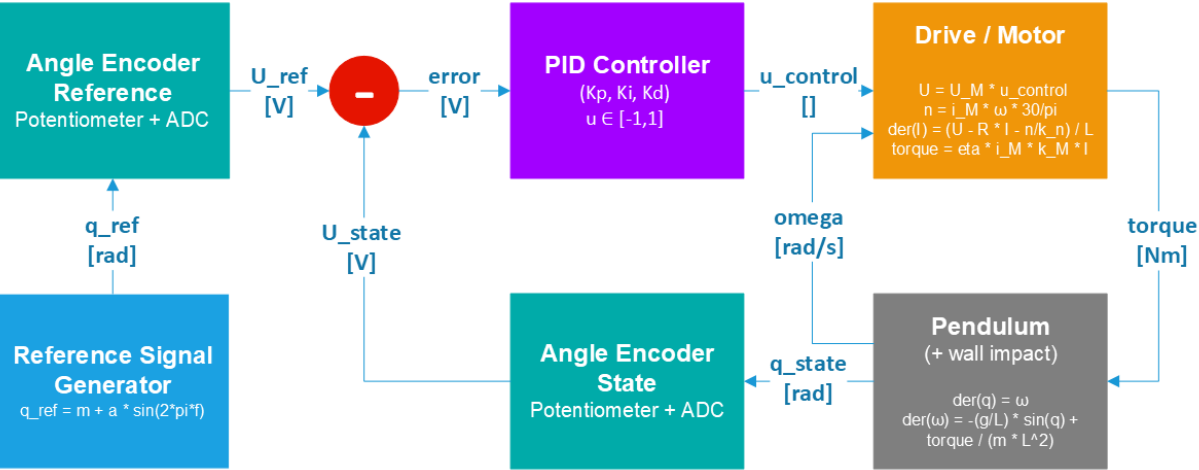
Core Layer:

- Core layer between the user definitions and the solver / model backends
- Provides a unified interface protocol for simulation components
- Simulation master orchestrates the execution of the subsystem models
- Configuration file is parsed, and corresponding system model is set up, connections are validated

Backend Layer:

- Layer represents the heterogeneous simulation engines
- Each backend is accessed exclusively through a wrapper, ensuring unified protocol compliance
- Equation-based models are built in OpenModelica and exports as Co-Simulation FMUs via OMPython
- FMUs are imported and simulated via FMPy (solver is inside the FMU)
- OpenSim models are simulated using opensim Python API
- FEM models could be solved via openCFS / CalculiX, interface via simulation file manipulation and subprocess execution (file-based adapters)
- **FEM: Proof of concept planned**

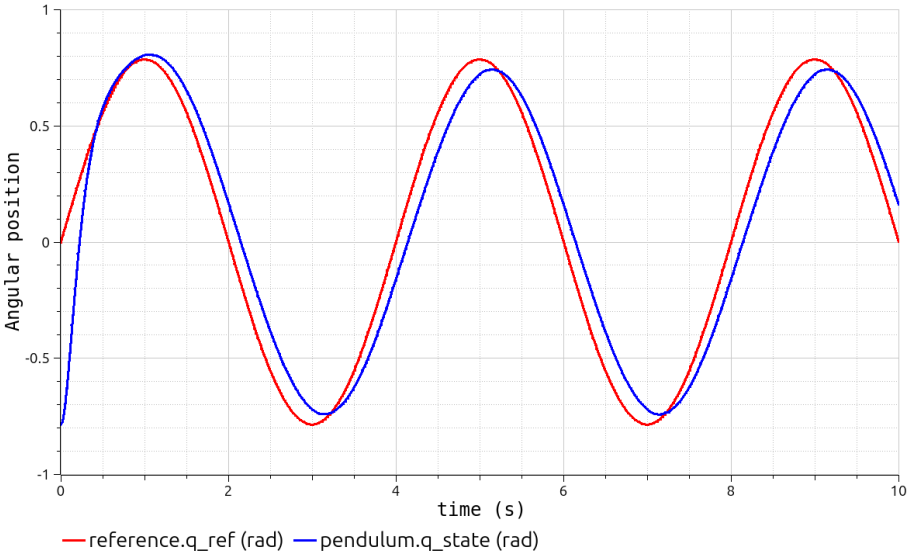
Simplified Block Diagram:



Suitability as Framework Demonstrator:

- Simple benchmark system with mechanical, electrical, and control subsystems
- Captures core challenges of system simulation:
 - Continuous + discrete dynamics
 - Sensors + quantization
 - Feedback control
 - Heterogeneous model integration
- Provides a progression of complexity:
 1. Equation-based pendulum + controller (FMUs)
 2. Hybrid extension (Impact Wall = discrete event)
 3. Replacement Pendulum with OpenSim model
 4. Integration with FEM (elastic rod/contact with wall)
- Serves as proof-of-concept for framework capabilities

Controlled Pendulum



Subsystem	Description
Reference Signal Generator	• Creates a sine signal oscillating around a mean with defined amplitude and frequency • Provides the reference input for the control loop (ideal trajectory)
Angle Encoder	• Maps the pendulum's angular position into an analog voltage using a potentiometer model • Adds a quantization via ADC with <i>nBits</i> resolution and clips to ADC voltage range
PID Controller	• Compares the reference signal (<i>U_ref</i>) and state signal (<i>U_state</i>) to compute an error • Implements the PID law (continuous and sample form implemented) • Outputs a control signal in the range -1 to 1 which drives the motor voltage
Drive / Motor	• Computes the current motor voltage based on the controller output • Computes the resulting current dynamics using inductance and resistance • Provides the actuation torque acting to the pendulum hinge joint
Pendulum	• Models a 1-DOF mechanical pendulum: rigid massless rod of length <i>L</i> with a point mass <i>m</i> at its end • Takes torque input from the drive, outputs angular position and velocity

(Sima, 2024)

Reasons for using OpenModelica and OMEdit:

- OpenModelica + OMEdit are used for equation-based subsystem modeling
- Modelica language provides:
 - Acausal equation-based modeling
 - Inputs and outputs of models can be defined irrespectively of equation formulation
 - Access to standard library: variety of standard component models from different engineering domains
- Packages allow modularity, provide an own namespace, and enable hierarchical composition of subsystems
- OpenModelica Compiler provides advanced features:
 - Dependency analysis and automatic ordering of equations
 - Symbolic manipulation and index reduction of DAEs
- Models can be exported as FMUs
- No need for reimplementatation of DAE handling and solving functionality

Subsystem Example: OpenModelica Pendulum Model

- Inputs and outputs are defined as **block interfaces** of type Real
 - **Input:** torque from drive that acts on the pendulum mechanicscs
 - **Outputs:** Angular position and angular velocity of the pendulum
- Parameters describe the properties and initial state of the model (can be changed by user)
- Initial equation and equations describe the behavior of the model
- **Acausal:** Outputs of the model (*q_state* and *omega_state*) are not required to be written on the LHS of the equation
- **Causality is automatically resolved** by compiler during simulation or FMU export

Example: Pendulum

```
1 within ControlledPendulum;
2
3 model Pendulum
4   /*
5    The model represents a simple 1-DOF pendulum model with a head with mass m at
6    a radial distance of L relative to the hinge joint center. The connection between
7    joint and head is considered as massless.
8    The state angle q_state is 0 for vertical alignment between head and joint center.
9    */
10
11   // Imports
12   import Modelica.Constants.pi;
13
14   // Parameters
15   parameter Real m(unit="kg") = 80*0.2;
16   parameter Real L(unit="m") = 0.4;
17   parameter Real q0(unit="rad") = -pi/4;
18   parameter Real omega0(unit="rad/s") = 0;
19
20   // Input torque at the pivot
21   Modelica.Blocks.Interfaces.RealInput torque(unit="N.m");
22
23   // Outputs
24   Modelica.Blocks.Interfaces.RealOutput q_state(unit="rad");
25   Modelica.Blocks.Interfaces.RealOutput omega_state(unit="rad/s");
26
27   protected
28     // Internal constants
29     constant Real g(unit="m/s^2") = 9.81;
30
31   initial equation
32     q_state = q0;
33     omega_state = omega0;
34
35   equation
36     der(q_state) = omega_state;
37     der(omega_state) = -(g / L) * sin(q_state) + torque/(m * L^2);
38 end Pendulum;
```

Example: Reference Model

```
1 within ControlledPendulum;
2
3 model Reference
4   // Constants
5   import Modelica.Constants.pi;
6
7   // Output
8   Modelica.Blocks.Interfaces.RealOutput q_ref(unit="rad");
9
10  // Parameters
11  parameter Real mean(unit="rad") = 0;
12  parameter Real amplitude(unit="rad") = pi/4;
13  parameter Real frequency(unit="Hz") = 0.25;
14
15  equation
16    q_ref = mean + amplitude * sin(2 * pi * frequency * time);
17  annotation( ... );
18
19 end Reference;
```

Example: Angle Encoder (Potentiometer + ADC) Model

```
within ControlledPendulum;

model AngleEncoder
  // Imports
  import Modelica.Constants.pi;

  // Input
  Modelica.Blocks.Interfaces.RealInput q(unit="rad");
  // Outputs
  Modelica.Blocks.Interfaces.RealOutput U_q(unit="V");

  // Public Parameters
  parameter Real nBits = 8 "ADC resolution [bits]";
  parameter Real q_min(unit="rad") = -pi/9;
  parameter Real q_max(unit="rad") = pi/9;

protected
  // Constants
  constant Real U_0_pot(unit="V") = 5;
  constant Real U_ADC(unit="V") = 3;
  constant Real R3(unit="Ohm") = 80e3;
  constant Real R4(unit="Ohm") = 20e3;
  constant Real alpha_max(unit="rad") = 3/4 * 2 * pi;

  // Internal Parameters
  parameter Real levels = (2^nBits - 1);
  parameter Real delta(unit="V") = U_ADC / levels;

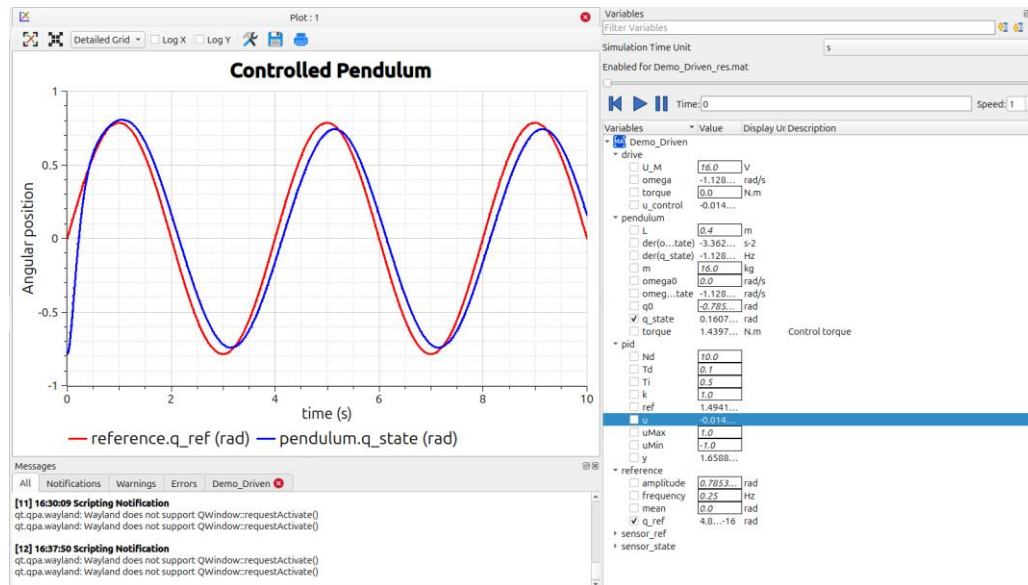
  // Internal Variables
  Real alpha(unit="rad");
  Real U_a(unit="V") ;

algorithm
  // Potentiometer mapping
  alpha := q - (q_min + q_max) / 2 + alpha_max/2;
  U_a := U_0_pot * (1 - R4 / (R3 + R4) - (alpha_max - alpha) / alpha_max);

  // Round to nearest code and map back to volts
  U_q := delta*integer(U_a / delta);
end AngleEncoder;
```

Building a System Model in Open Modelica:

- Subsystems (Reference, Sensors, Drive, PID, Pendulum) combined into a **system model package**
- **Components can be instantiated** within the system model
- Variables which are defined as **parameters can be set for each component**
- **Connect equations** are used to describe the dependencies
- **OpenModelica resolves the total resulting system of equations**
- OMEdit editor supports **hierarchical composition** by connecting subsystem containing other components
- Full **system model can be simulated directly in OpenModelica** for validation before FMU export
- All components are Modelica-based, **no external coupling yet**
- **Parameters can be changed without recompilation**



Example: Driven Demo System Model

```
1 within ControlledPendulum;|
2
3 model Demo_Driven
4   // Components
5   Reference          reference(frequency=0.25);
6   AngleEncoder       sensor_ref( q_min = -reference.amplitude,
7                                   q_max = reference.amplitude);
8   AngleEncoder       sensor_state( q_min = -reference.amplitude,
9                                    q_max = reference.amplitude);
10  PID_Continuous     pid;
11  Drive              drive;
12  Pendulum           pendulum;
13
14  equation
15    // Connect reference and pendulum to sensors
16    connect(reference.q_ref, sensor_ref.q);
17    connect(pendulum.q_state, sensor_state.q);
18
19    //connect(sensor_ref.U_q, pid_sensors.reference);
20    connect(sensor_ref.U_q, pid.ref);
21    connect(sensor_state.U_q, pid.y);
22
23    // Connect PID output u_control to drive
24    connect(pid.u, drive.u_control);
25
26    // Connect drive and Pendulum
27    connect(drive.torque, pendulum.torque);
28    connect(drive.omega, pendulum.omega_state);
29
30    annotation( ... );
31
32
33
34 end Demo_Driven;
```


Example: Driven Demo System Model using OMPython

```
from path import Path
from OMPython import ModelicaSystem

# Define paths and package information
root = Path(r'/home/flo/repos/SystemSimulation/demos/ControlledPendulum/')
pkg_str = str(root / "ControlledPendulum" / "package.mo")
pkg_name = "ControlledPendulum"
model_name = pkg_name + "." + "Demo_Driven"

# Create and build the model
demo_model = ModelicaSystem(pkg_str, model_name)
demo_model.buildModel()

# Change model parameters
demo_model.setParameters("reference.frequency=0.33")

# Simulation parameters
current_options = demo_model.getSimulationOptions()
print(f"Simulation options:", current_options)

# Run simulation
demo_model.simulate()

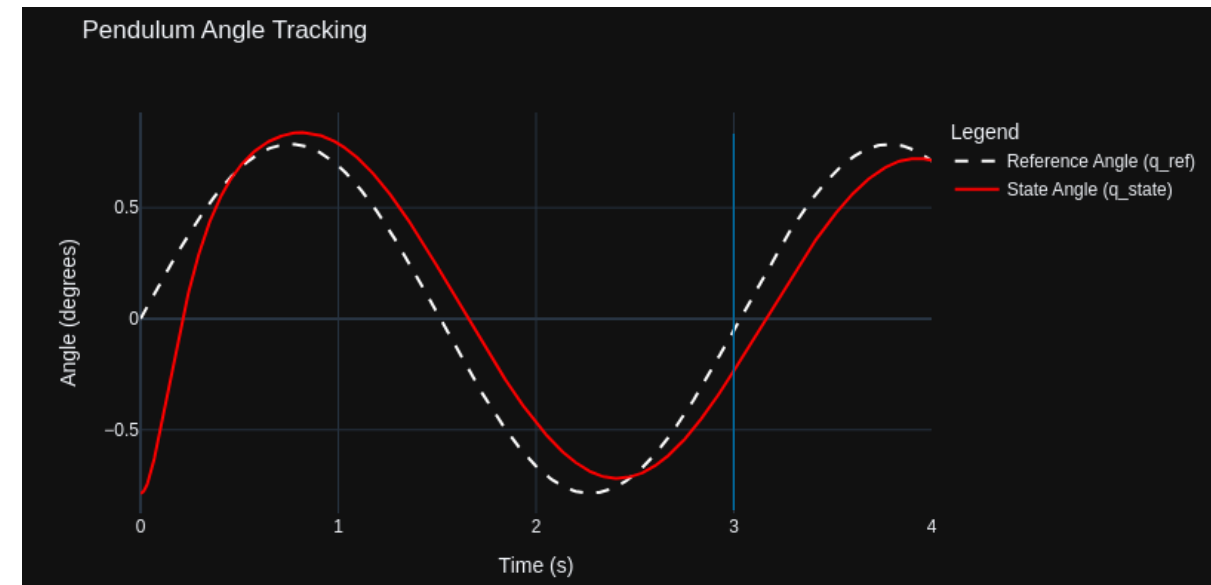
# Extract results
results = demo_model.getSolutions()
time_values = demo_model.getSolutions('time').flatten()
ref_values = demo_model.getSolutions('reference.q_ref').flatten()
state_values = demo_model.getSolutions('pendulum.q_state').flatten()
```

Simulation options:
startTime: 0.0
stopTime: 4.0
stepSize: 0.01
tolerance: 1e-6
solver: dassl
outputFormat: mat

Accessing OpenModelica Model via OMPython:

- OMPython: Interface between OpenModelica (OpenModelica Compiler) and Python
- **Wrapper library** that enables Python scripts to compile, simulate, and analyze Modelica models
- **Model Compilation** converts .mo files into executable simulation code
- **Parameter manipulation:** model parameters can be modified before simulation
- **Result Extraction:** time-series data can be obtained from simulation outputs
- **Batch Processing:** Allows to automate multiple simulation runs with different parameters

Example: Reference Trajectory with modified frequency $T = 3$ s



FMI Standard

- FMI-Standard defines an open interface, to be implemented by a FMU (Functional Mock-up Unit)
- FMU may be self-integrating (**Co-Simulation**) or require the master tool (importer) to perform the integration (**Model Exchange**)
- FMI **separates the description of the interface (xml)** from its **functionality (C API)**
- FMI standardizes
 - set of C-functions, to be implemented by a model/simulator
 - an **XML-model description** file to be provided by a model/simulator
 - the distribution file format to be used by a model/simulator

Advantages of FMI / FMUs:

- **Enables heterogeneity:** Modelica Models can be used together with OpenSim models (FEM models)
- **Encapsulates internal model details** and exposes only required simulation interface

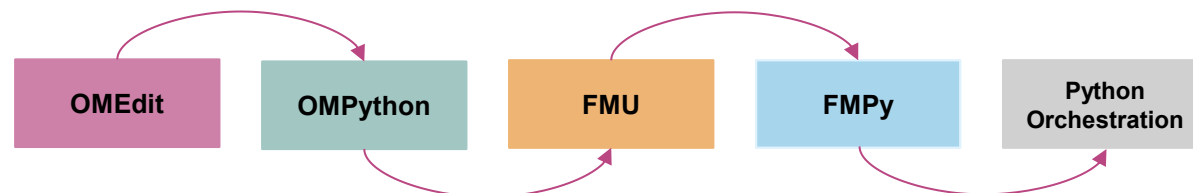
Functional Mock-up Unit

- **FMU is a zip-file** with defined structure
- **modelDescription.xml:** declares variables, inputs/outputs, parameters, units
- **binaries/:** compiled solver and model binaries per platform
- **sources/:** source code for building the FMU for other platforms
- **resources/:** additional data
- **documentation/:** icons, additional docs

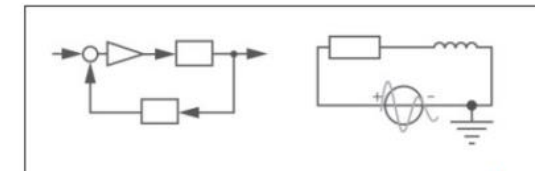
```
├── binaries
│   └── linux64
├── modelDescription.xml
├── resources
│   └── ControlledPendulum_AngleEncoder_JacD.bin
└── sources
    ├── CMakeLists.txt
    ├── CMakeLists.txt.in
    ├── config.guess
    ├── config.sub
    ├── configure
    ├── ControlledPendulum_AngleEncoder_01exo.c
    ├── ControlledPendulum_AngleEncoder_02nls.c
    ├── ControlledPendulum_AngleEncoder_03lsy.c
    ├── ControlledPendulum_AngleEncoder_04set.c
    ├── ControlledPendulum_AngleEncoder_05evt.c
    ├── ControlledPendulum_AngleEncoder_06inz.c
    └── ControlledPendulum_AngleEncoder_07dly.c
```

Workflow: From Modelica to FMU to Python

1. **Subsystem Modeling** in OpenModelica via OMEdit
2. **Build/compile** models with OMPython
3. **Export** Subsystems as CS-FMUs
4. **Move FMUs** to project directory (target destination)
5. **Import and Simulate** with FMPy (Python library for FMI 1/2/3)

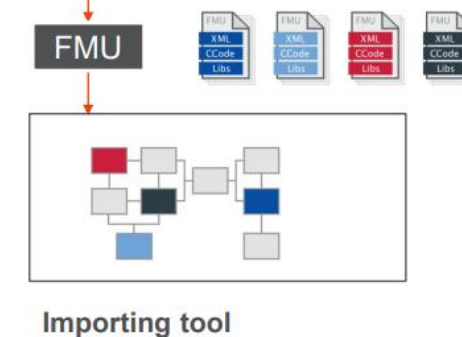


Exporting tools



1. develop model
2. export as **Functional Mockup Unit (FMU)** according to the **FMI Standard**

3. import FMU
4. (co-) simulate model



(Modelica Association, 2022)

1. Read Model Description & Build VR Map

```
from fmpy import read_model_description

# Read Model Description
path = fmu_paths['Pendulum']
md = read_model_description(path)
variables = md.modelVariables
model_variables = {}
for variable in variables:
    temp_dict = {}
    temp_dict['name'] = variable.name
    temp_dict['unit'] = variable.unit
    temp_dict['ValueReference'] = variable.valueReference
    temp_dict['causality'] = variable.causality
    temp_dict['variability'] = variable.variability
    model_variables[variable.name] = temp_dict
```

FMU: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs/Pendulum.fmu

Variable Name	ValueRef	Causality	Variability	Unit
D_outputAlias_omega_state	0	local	continuous	rad/s
D_outputAlias_q_state	1	local	continuous	rad
der(D_outputAlias_omega_state)	2	local	continuous	s-2
der(D_outputAlias_q_state)	3	local	continuous	Hz
omega_state	6	output	continuous	rad/s
q_state	7	output	continuous	rad
torque	8	input	continuous	N.m
L	9	parameter	fixed	m
m	10	parameter	fixed	kg
omega0	11	parameter	fixed	rad/s
q0	12	parameter	fixed	rad

```
# Dictionary to hold variable references
vars_rf = {}
vars_rf['q_ref'] = fmu_dict['Reference']['ValueRefs']['q_ref']

vars_rf['q_state'] = fmu_dict['Pendulum']['ValueRefs']['q_state']
vars_rf['omega_state'] = fmu_dict['Pendulum']['ValueRefs']['omega_state']
vars_rf['torque'] = fmu_dict['Pendulum']['ValueRefs']['torque']

vars_rf['q_sensor'] = fmu_dict['AngleEncoder']['ValueRefs']['q']
vars_rf['U_q'] = fmu_dict['AngleEncoder']['ValueRefs']['U_q']

vars_rf['pid_y'] = fmu_dict['PID.Continuous']['ValueRefs']['y']
vars_rf['pid_ref'] = fmu_dict['PID.Continuous']['ValueRefs']['ref']
vars_rf['pid_u'] = fmu_dict['PID.Continuous']['ValueRefs']['u']

vars_rf['drive_u'] = fmu_dict['Drive']['ValueRefs']['u_control']
vars_rf['drive_omega'] = fmu_dict['Drive']['ValueRefs']['omega']
vars_rf['drive_torque'] = fmu_dict['Drive']['ValueRefs']['torque']
```

- Access to FMU properties such as model variables
- Causality, Variability, and Unit attributes allow connection validation
- Value references are required to get/set variables of the FMU

2. Instantiate, Setup, and Initialize

```
from fmpy import extract
from fmpy.fmi2 import FMU2Slave

# Instantiate Pendulum
guid = fmu_dict['Pendulum']['ModelDescription'].guid
unzip_dir = extract(fmu_paths['Pendulum'])
modelIdentifier = md.coSimulation.modelIdentifier
pendulum_fmu = FMU2Slave(guid=guid,
                          unzipDirectory=unzip_dir,
                          modelIdentifier=modelIdentifier,
                          instanceName='pendulum_fmu')
pendulum_fmu.instantiate()
```

```
for fmu in fmu_list:
    fmu.reset()
    fmu.setupExperiment(startTime=0.0)
    fmu.enterInitializationMode()
    # Set parameters
    # if fmu is ref fmu:
    #     fmu.setReal([params_rf['ref_mean'],
    #                  # params_rf['ref_amplitude'],
    #                  # params_rf['ref_frequency']],
    #                  [0.0, 1.0, 0.2])
    fmu.exitInitializationMode()
```

- FMUs are instantiated as FMU2Slave instances identified by a global unique ID (guid)
- FMUs are initialized by calling the **setupExperiment** method defining the start time, ensures that all FMUs start
- The **enterInitializationMode** and **exitInitializationMode** methods are called to complete the initialization process
- Within the initialization mode, initial values for input variables can be set using the **set[Type]** method
- Note:**
 - Default Initial values and parameters defined in the Modelica models are used
 - Default solver settings are used (e.g., tolerances)

3. Master Loop

```
while t < tf:
    # 1) Read the current reference position
    ref_fmu.doStep(t, h)
    q_ref = ref_fmu.getReal([vars_rf['q_ref']])

    # 2) Read the plant state at the current time
    q_state = pendulum_fmu.getReal([vars_rf['q_state']])
    omega_state = pendulum_fmu.getReal([vars_rf['omega_state']])

    # 3) Sensors: set inputs -> step -> read outputs
    sensor_ref_fmu.setReal([vars_rf['q_sensor']], q_ref)
    sensor_state_fmu.setReal([vars_rf['q_sensor']], q_state)
    sensor_ref_fmu.doStep(t, h)
    sensor_state_fmu.doStep(t, h)
    U_q_ref = sensor_ref_fmu.getReal([vars_rf['U_q']])
    U_q_state = sensor_state_fmu.getReal([vars_rf['U_q']])

    # 4) Controller: set inputs -> step -> read outputs
    pid_fmu.setReal([vars_rf['pid_y'], vars_rf['pid_ref']],
                    [U_q_state[0], U_q_ref[0]])
    pid_fmu.doStep(currentCommunicationPoint=t, communicationStepSize=h)
    u_pid = pid_fmu.getReal([vars_rf['pid_u']])

    # 5) Drive: set inputs -> step -> read outputs
    drive.setReal([vars_rf['drive_u'], vars_rf['drive_omega']],
                  [u_pid[0], omega_state[0]])
    drive.doStep(currentCommunicationPoint=t, communicationStepSize=h)
    torque = drive.getReal([vars_rf['drive_torque']])

    # 6) Plant: set inputs -> step
    pendulum_fmu.setReal([vars_rf['torque']], torque)
    pendulum_fmu.doStep(currentCommunicationPoint=t, communicationStepSize=h)

    # Log data
    ts.append(t)
    q_ref_log.append(q_ref[0])
    q_state_log.append(q_state[0])
    omega_state_log.append(omega_state[0])
    U_ref_log.append(U_q_ref[0])
    U_state_log.append(U_q_state[0])

    # Advance time
    t += h
```

- The simulation loop iterates over the defined time steps, advancing the simulation time by the specified step size
- Here: constant and uniform time steps for all involved FMUs
- fmu.getReal([vr]):** Retrieves the current value of a real variable identified by its value reference (VR).
- fmu.setReal([vr], [value]):** Sets the value of a real variable identified by its VR.
- fmu.doStep(currentCommunicationPoint, communicationStepSize):** Advances the simulation by a specified step size from the current communication point.

4. Terminate & Free Instance

```
for fmu in fmu_list:
    try:
        fmu.terminate()
    finally:
        fmu.freeInstance()
```

- FMU Cleanup and resource management
- Required since FMUs wrap native libraries and manage memory
- fmu.terminate():** signals the FMU to perform its internal cleanup operations
- fmu.freeInstance():** performs the final resource deallocation by calling **fmi2FreeInstance()** and **freeLibrary()**

Summary Key Points:

- All FMUs share a single master time
- Get/set methods require iterables arguments and provide batch getting and setting of variables
- Causality: write to input parameters, read from output parameters
- Parameters are set during initialization
- Events & Rollbacks:
 - FMI 2.CS FMUs cannot rollback, h must be small to prevent event overstepping, but report an early return
- Current Demo does not contain discrete events

OpenSim Pendulum Model Setup

Body & Joint:

- Pendulum head modeled as a rigid body with defined mass and inertia
- Connected to ground via a **PinJoint** (rotation about z-axis)
- **Coordinate q** represents the angular displacement

Actuator

- **CoordinateActuator** applied to joint coordinate q
- Input: torque command (control signal = torque)
- Output: torque applied directly at the joint
- Actuation overriding must be enabled

Controller

- **PrescribedController** with a Constant function
- Control signal is updated each simulation step via Python

```
# Define coordinate
coord = joint.updCoordinate()
coord.setName('q')
coord.setDefaultValue(0)
coord.setDefaultSpeedValue(0)

# Define CoordinateActuator
act = osim.CoordinateActuator()
act.setName('torque')
act.setCoordinate(coord)
act.setOptimalForce(1)
act.setMinControl(-1e6)
act.setMaxControl(1e6)
model.addForce(act)

# Define controller
func = osim.Constant()
ctrl = osim.PrescribedController()
ctrl.setName('controller')
ctrl.addActuator(act)
ctrl.prescribeControlForActuator('torque', func)
model.addController(ctrl)
```

```
while t < tf:
    if t < 0.25 * tf:
        act.setOverrideActuation(state, -0.05)
    elif t < 0.75 * tf:
        act.setOverrideActuation(state, 0.05)
    else:
        act.setOverrideActuation(state, -0.05)

    model.realizeDynamics(state)
    model.realizeAcceleration(state)

    state = manager.integrate(t + dt)
    model.realizePosition(state)
    model.realizeVelocity(state)
    model.realizeAcceleration(state)
    model.realizeDynamics(state)

    t = state.getTime()
```

Simulation Workflow

Initialization

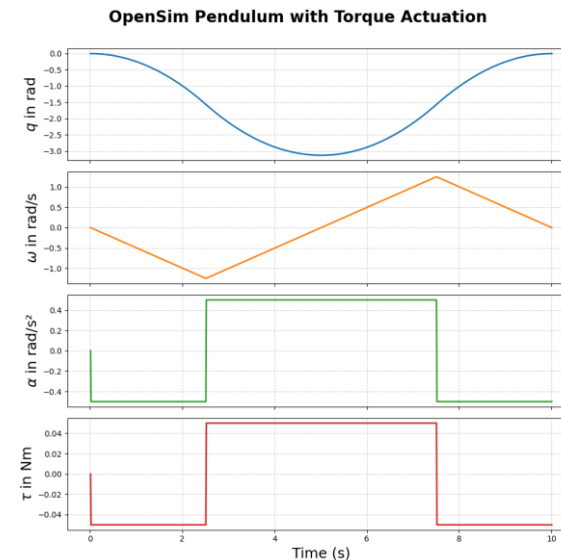
- When the model is initialized and built a reference to the writable **model state** is obtained
- **Simulation Manger** object handles the simulation & integration of the OpenSim model instance and is **initialized with the model state**

Torque Application

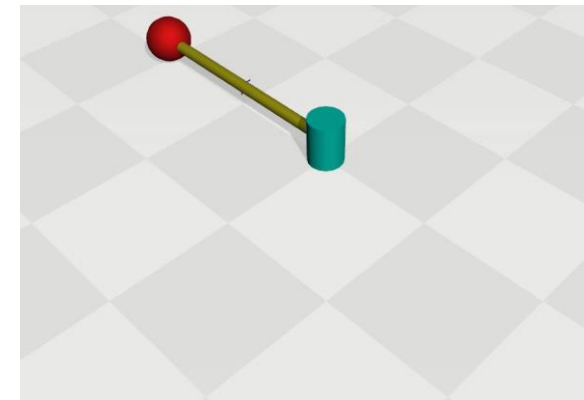
- Torque is applied by setting the actuation of the coordinate actuator using **actuator.setOverrideActuation(state, torque)**
- The torque is made effective (acting on the model) by calling the methods **model.realizeDynamics(state)** and **model.realizeAcceleration(state)** in this order

Performing the simulation step

- The new simulation step (new model state) is obtained by calling **manager.integrate(t + dt)**
- New position, velocity, acceleration, and dynamics are made effective by calling the **realize methods** on the mode with the new state as argument



Visualization of the Resulting Motion (openSim Creator)



Goal:

- Treat any simulator (FMU, OpenSim, future FEM) as a CoSimulation component
- Attempt to take heterogeneous runtimes (FMU / FMPy) and opensim and **make them look the same for the simulation master**
- CoSimulationComponent Protocol defines a **behavioral interface for the different types**
- **Configuration file** is used to define configuration of involved models (FMUs / OpenSim)
- Depending on type, models are instantiated as FMUComponent or OpenSimComponent which implement the CoSimulation Component Protocol

```
1 from typing import Protocol, Dict, Any, runtime_checkable
2
3 @runtime_checkable
4 class CoSimComponent(Protocol):
5     """
6     Minimal interface for a co-simulation component (FMU, OpenSim model, etc.).
7     Implementations do not need to inherit from this class,
8     but must implement the methods defined here (structural subtyping).
9     """
10     name: str
11
12     def initialize(self, t0: float) -> None:
13         """Prepare the component for simulation starting at time t0."""
14         ...
15
16     def set_parameters(self, **parameters: Any) -> None:
17         """Set the parameters of an FMU during initialization."""
18         ...
19
20     def set_inputs(self, **signals: float) -> None:
21         """Set the input signals for the next step."""
22         ...
23
24     def step(self, t: float, h: float) -> None:
25         """Advance the component's state from time t to t+h."""
26         ...
27
28     def get_outputs(self) -> Dict[str, float]:
29         """Return the current outputs as a dictionary {name: value}."""
30         ...
31
32     def reset(self) -> None:
33         """Reset the component to a clean state before (before initialization)."""
34         ...
```

FMU Configuration:

- **Configuration file** is used to define configuration of involved models (FMUs / OpenSim)
- **Currently:**
 - Global simulation time settings
 - Paths to FMUs
 - Input / output variable mapping
- **Future:**
 - Enable also OpenSim / FEM model configuration
 - Allow definition of model parameters and initial values

```
# Configuration for Controlled Pendulum Hybrid Simulation Demo
step:
  t0: 0.0
  tf: 10.0
  h: 0.001

fmus:
  Reference:
    path: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs
    inputs: {}
    outputs: {q_ref: "q_ref"}

  SensorRef:
    path: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs
    inputs: {q: "q"}
    outputs: {U_q: "U_q"}

  SensorState:
    path: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs
    inputs: {q: "q"}
    outputs: {U_q: "U_q"}

  PID:
    path: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs
    inputs: {ref: "ref", y: "y"}
    outputs: {u: "u"}

  Drive:
    path: /home/flo/repos/SystemSimulation/demos/ControlledPendulum/FMUs
    inputs: {u_control: "u_control", omega: "omega"}
    outputs: {torque: "torque", omega: "omega"}
```

fmu_component.py:

- Simplified **high-level interface** around FMPy's low-level FMU operations
- **Single step initialization** handles all required FMPy steps internally
- **Automatic mapping of signal names** to FMU value references
- Handles signals for setting and getting via keyword arguments instead of manual value reference lookup

opensim_pendulum.py

- Provides simplified interface around OpenSim Python API
- Handles entire model building and setup
- Single method call for simulation replaces complex OpenSim workflow
- Automatic realization of all levels (position, velocity, acceleration, dynamics)
- State consistency: model realization after every parameter or input change
- Integration coordination: Manager and state object are kept synchronized

Scheduler for Controlled Pendulum Demo:

- FMUComponent handles value references for setting inputs and getting outputs internally
- OpenSim pendulum encapsulates model state, model updates, and manager internally
- Unified interaction for FMU models and OpenSim pendulum model

```
while t < tf - 1e-12:
    # 1) Get reference signal
    ref.step(t, h)
    q_ref = ref.get_outputs()[q_ref]

    # 2) Get plant outputs at current time
    plant_out = plant.get_outputs()
    q_state, omega_state = plant_out[q], plant_out[omega]

    # 3) Feed reference and state to sensor models
    sensor_ref.set_inputs(q=q_ref)
    sensor_state.set_inputs(q=q_state)

    sensor_ref.step(t, h)
    sensor_state.step(t, h)

    U_q_ref = sensor_ref.get_outputs()[U_q]
    U_q_state = sensor_state.get_outputs()[U_q]

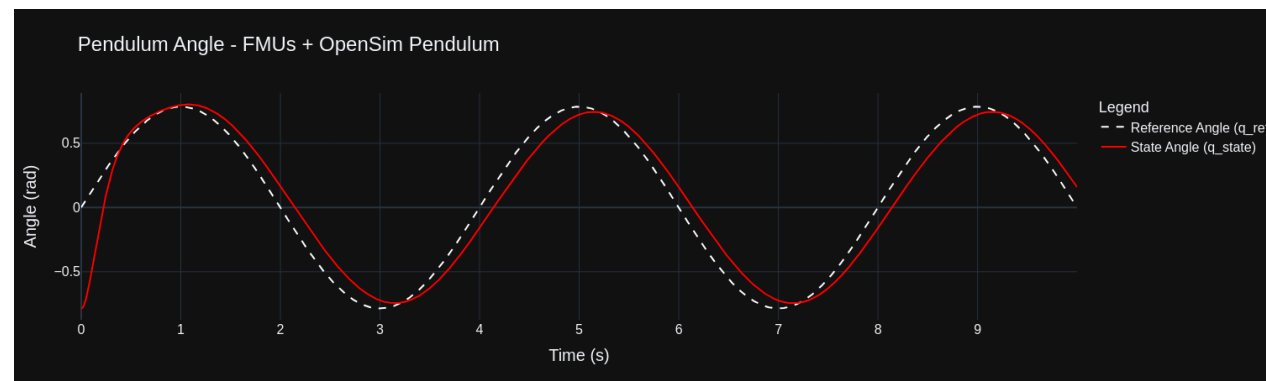
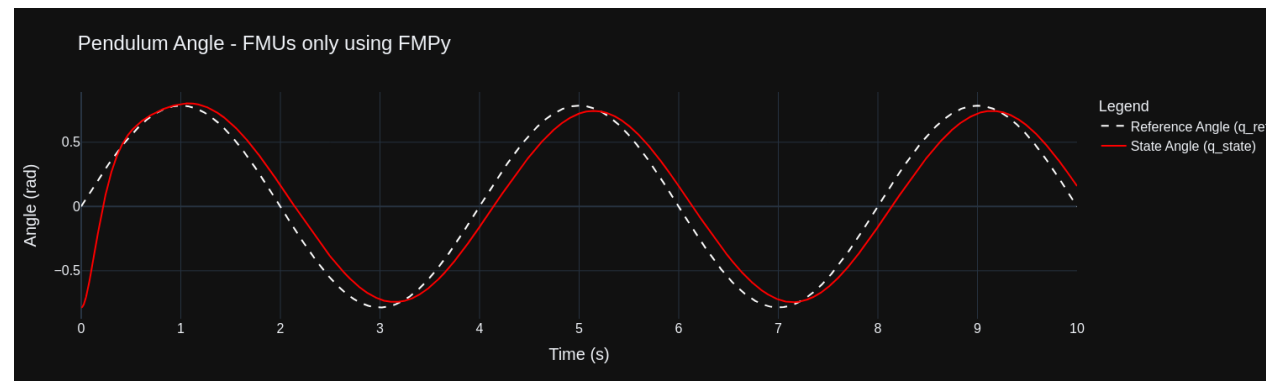
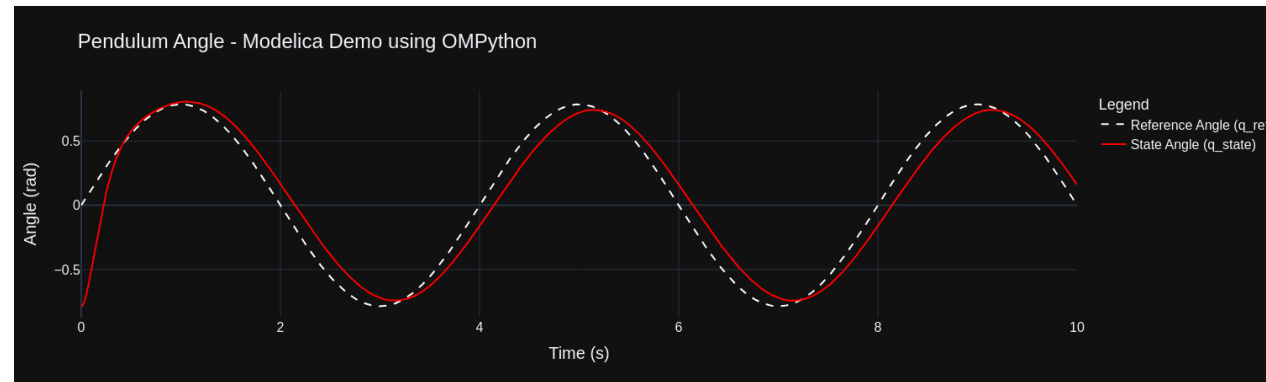
    # 4) Feed sensor outputs to PID controller
    pid.set_inputs(ref=U_q_ref, y=U_q_state)
    pid.step(t, h)
    u_pid = pid.get_outputs()[u]

    # 5) Drive
    drive.set_inputs(u_control=u_pid, omega=omega_state)
    drive.step(t, h)
    torque = drive.get_outputs()[torque]

    # 6) Apply torque to plant and step
    plant.set_inputs(torque=torque)
    plant.step(t, h)

    if logger:
        logger(t, dict(
            q_ref=q_ref, q_state=q_state, omega_state=omega_state,
            U_q_ref=U_q_ref, U_q_state=U_q_state,
            u_pid=u_pid, torque=torque
        ))

    t += h
```



Motivation for Wall Impact Modeling:

- Wall impact **adds hybrid model behavior** (continuous dynamics + discrete events)
- Allows **integration of different modeling approaches** (eq.-based, biomechanical, FEM)
- **Hybrid effect:** Adds non-smooth dynamics (impact, bounce)
- **Introduces discrete events** (zero-crossings at wall position)
- **Physically intuitive and simple** to explain (pendulum hits a wall)
- Extends controlled pendulum **demo from “continuous only” to continuous–discrete hybrid**
- Provides **a natural test case for event handling**
- **Allows higher-fidelity FEM contact models**

Modelica: Discrete wall model (event-based, when/reinit)

- Impact detected when pendulum angle $\leq$ wall angle.
- At event: velocity sign flipped & scaled by restitution coefficient e
- Discontinuous jump: requires event and state reinitialization
- Shows classic hybrid behavior (ODE + discrete event)
- Simpler, but less physical (no continuous penetration force)
- Can be used for demonstrating event handling in master.

equation

```
when {q_state > q_wall} then
  reinit(omega_state, -e * pre(omega_state));
end when;
```

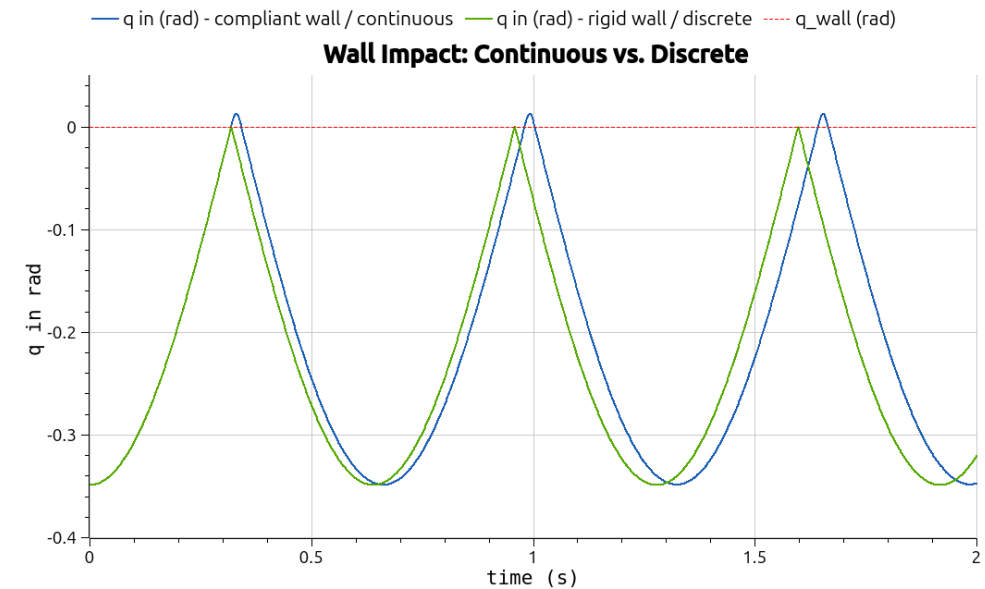
Modelica: Continuous Wall model / Compliant Contact

- Wall modeled as spring–damper with stiffness k and damping c
- Contact torque grows smoothly with penetration (regularized ReLU + ϵ)
- Captures continuous force evolution (like Hunt–Crossley)
- Suitable for CS-FMU export and integration into system model
- Advantage: no hard event handling, smoother integration.

```
// Compression-only rate (noEvent to avoid superfluous events)
comp = noEvent( max(pen_dot, 0) );

// Hunt–Crossley-like compliant torque (n=1): k*pen + c*pen*comp
// Negative sign: torque resists penetration (acts to reduce q when x>0)
torque = -( k*pen + c*pen*comp );

// Boolean contact flag without generating events
contact = noEvent( q > q_wall );
```



OpenSim: Hunt Crossley Contact

- Contact modeled as compliant sphere–plane contact
- Directly supported in OpenSim via **HuntCrossleyForce**
- Natural biomechanics connection
- Handles **continuous impact with damping**
- Demonstrates ability to replace eq.-based pendulum with biomechanical one
- Shows hybrid integration: OpenSim + FMU in one loop
- **Limitation:** more complex parameterization (contact geometry, stiffness, damping)
- **Advantage:** can export motion files for visualization in OpenSim GUI.

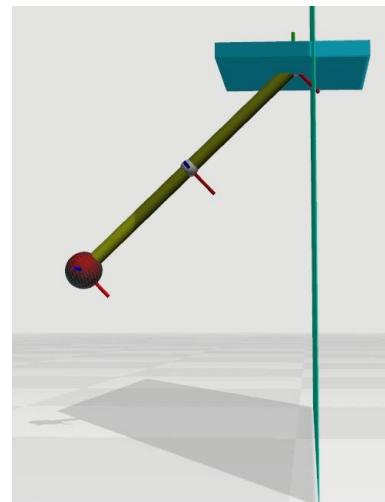
PrePoMax / CalculiX (FEM impact)

- **Static Analysis:**
 - Hertz contact (sphere–plane, hard formulation).
 - Extracts stiffness curve (force vs. penetration).
 - Allows stress/strain fields in contact region
- **Dynamic Analysis:**
 - Implicit Dynamic Simulation
 - Surface interaction: linear-pressure overclosure
- Shows true advantage of FEM integration: **localized stresses and deformations**
- But: limited Python scripting in PrePoMax, integration not trivial
- **Candidates for future:**
 - CalculiX standalone with Python editing of input decks
 - openCFS (no implicit contact modeling, available via Nitsche Interface Non-Conforming Interface)

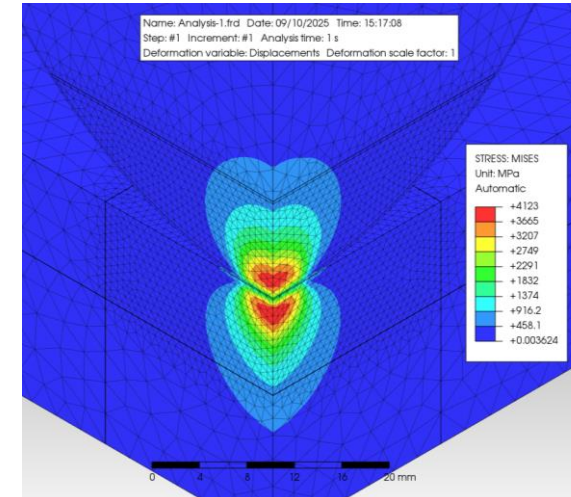
HuntCrossleyForce Definition

Properties	
actions (?)	
name (?)	HuntCrossleyForce
components (?)	(No Objects)
appliesForce (?)	☒
geometry (?)	wall_contact  head_contact  + Add Entry
stiffness	100000.000000 - +
dissipation	0.500000 - +
static_friction	0.900000 - +
dynamic_friction	0.900000 - +
viscous_friction	0.600000 - +
transition_velocity (?)	0.010000 - +

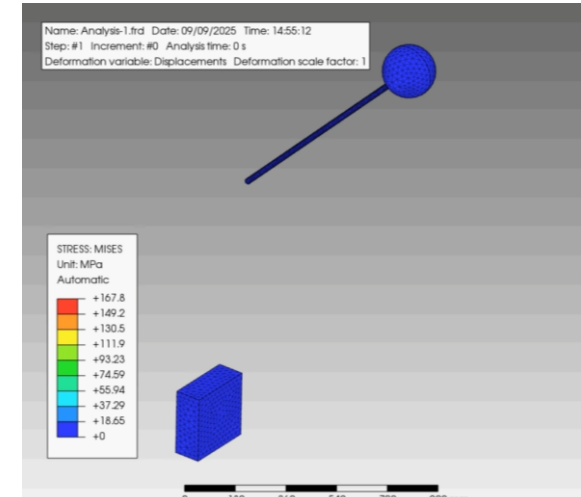
OpenSim Visualization



Static: Hertzian Contact



Dynamic Impact



ID	User Requirement	Status	Justification	Further Improvements
UR-1	The framework shall support the component-based modeling of multi-domain systems .	Partially	- Controlled Pendulum Demo with separate subsystems - Modelica packages allow modular composition Multi-domain aspects covered (mechanical + electrical + control)	Define generic interfaces for plug-and-play across domains
UR-2	The framework shall support the use of units for all physical quantities .	Partially	Modelica subsystems use SI units consistently (checked by OpenModelica compiler) - FMUs expose units in modelDescription.xml OpenSim API provides units for coordinates and forces	- Implement automated unit compatibility checks when connecting FMUs/OpenSim models - Standardize unit handling inside the framework - Provide clear error/warning messages on mismatched units
UR-3	The framework shall allow the creation of system models by connecting subsystems .	Fulfilled	- Controlled Pendulum Demo: subsystems (Reference, Sensor, PID, Drive, Pendulum) connected into a full system in Modelica Exported subsystems as FMUs and orchestrated connections manually in Python using FMPy - Demonstrated successful data exchange between connected FMUs	- Provide YAML/JSON-based system configuration for easier model assembly Automate connection handling and variable mapping (reduce manual coding) - Extend connection mechanism to handle event-driven interactions
UR-4	The framework should allow the acausal equation-based modeling of the component behavior.	Fulfilled	OpenModelica + OMEdit used for subsystem modeling (Drive, Pendulum, PID, Sensor, Reference) Equations expressed acausally (no manual input–output assignment required) OpenModelica compiler resolves causality automatically (DAE handling, index reduction) - Exported as FMUs for integration in Python	- Document best practices for equation-based subsystem creation (templates, examples) - Add library of reusable Modelica templates for common mechatronic components - Consider hybrid extension: support for acausal + event-driven (discrete) subsystems
UR-5	The framework shall enable the causal equation-based modeling of the component behavior.	Fulfilled	- Some subsystems (e.g., Sensor, PID) already modeled causally with explicit input/output ports - Co-Simulation FMUs (from Modelica, OpenSim) expose I/O interfaces by design - Controlled Pendulum Demo: causal signal flow in the orchestration layer	- Provide clear guidelines/templates for causal subsystem modeling within the framework - Add automated checks for I/O compatibility when connecting causal subsystems
UR-6	The framework shall support the definition of component behavior using discrete-time or hybrid dynamics .	Partially	- Discrete PID controllers implemented in Modelica (sampled when sample(...)) - ImpactWall modeled both as continuous contact dynamics and discrete event (when/reinit) in Modelica - OpenSim Hunt–Crossley contact provides hybrid continuous–discrete behavior (force + event-like contact activation) - Python orchestration loop handles stepwise updates	Extend orchestration with event handling & rollback for hybrid FMUs - Provide examples beyond the pendulum (e.g., switching controllers, digital sensors)

ID	User Requirement	Status	Justification	Further Improvements
UR-7	The framework should provide templates for the creation of equation-based component models .	Open	- Current subsystem models (Pendulum, Drive, PID, Sensor, Reference) serve as examples but not as reusable templates - No formalized template library exists yet in the framework Users must currently rely on OMEdit + Modelica standard library	Develop template models (e.g., mechanical body, motor, sensor, controller) in Modelica within a dedicated package Provide documentation + guidelines for extending templates Integrate template generation into framework workflow (e.g., pre-packaged FMUs for common components)
UR-8	The framework shall allow the reuse of components and subsystems with different configurations.	Partially	Subsystems in Modelica (e.g., PID, Drive, Sensor) are parameterized and can be reused with different settings FMUs can be instantiated multiple times with different parameter values - Demo models already reused the Sensor for reference and state signals	Provide systematic configuration management (e.g., YAML/JSON config files) - Add support for saving/loading predefined subsystem configurations

ID	User Requirement	Status	Justification	Further Improvements
UR-9	The framework shall enable the integration of subsystem models developed using different modeling tools .	Partially	FMUs exported from OpenModelica integrated via FMPy OpenSim pendulum model integrated alongside FMUs in same co-simulation loop - Demonstrated heterogeneous integration (FMUs + OpenSim) in Controlled Pendulum demo	- Extend integration to FEM solvers (e.g., CalculiX, openCFS) - Provide standardized wrapper interfaces for all external tools Automate toolchain handling (compilation, FMU export, setup)
UR-10	The framework should support the combination of equation-based and other heterogeneous sub models .	Partially	Equation-based subsystems modeled in Modelica and exported as FMUs Combined with OpenSim pendulum model in one co-simulation loop Python master orchestrates data exchange consistently	Formalize interface contracts between equation-based FMUs and heterogeneous models (OpenSim, FEM) Improve event handling for hybrid interactions (e.g., contact, switching controllers) - Demonstrate larger heterogeneous systems beyond pendulum case

ID	User Requirement	Status	Justification	Further Improvements
UR-11	The framework shall enable the numerical simulation of hybrid systems with both continuous and discrete-time behavior.	Partially	• Discrete PID controllers modeled in Modelica (sampled dynamics) • ImpactWall implemented in two variants: • continuous compliant model (stiffness + damping) • discrete event-based model (when + reinit) • OpenSim Hunt–Crossley contact adds hybrid force law • Co-simulation loop executes continuous + discrete subsystems together	• Implement robust event-handling and rollback in the Python master • Demonstrate multi-event scenarios (e.g., multiple collisions, switching control laws) • Generalize framework support for hybrid FMUs
UR-12	The framework shall allow the user to configure and select solver types and step sizes for simulation.	Partially	• Global master communication step size h is user-set in your Python loop (FMPy) • OpenSim side: integrator accuracy / step can be set via the Manager/Integrator • CS-FMUs bring their internal solvers • step size and tolerances can be influenced via doStep size and via FMU parameters (when exposed).	• Add a uniform config layer (YAML) for: global master step, per-component step sizes, synchronization (fixed/adaptive) • OpenSim integrator type (e.g., Runge–Kutta, implicit) and accuracy/step controls • Implement per-component rate (multi-rate co-simulation)

ID	User Requirement	Status	Justification	Further Improvements
UR-13	The framework should provide a visual representation of the system model and its hierarchical structure .	Open	- OMEdit already shows package hierarchy and block diagrams for the Modelica side No automated system graph yet from the Python framework	- Add a graph builder in SysSimX that reads the YAML config + FMU/OpenSim metadata and creates a hierarchical graph Export to graph for PNG/SVG and - Support: system → subsystem → component and port-level edges. - Overlay units, rates (sample times), and domains (mechanical/electrical/biomech/FEM) with color coding
UR-14	The framework should allow the user to observe simulation progress through graphical and textual output.	Partially	- Python co-simulation loop already prints stepwise logs (t, u, torque, states). - FMPy / FMU APIs provide status + error logs - OpenSim side: print manager/integrator accuracy and states.	- Add a unified Logger & Progress module (structured logs, tqdm progress bar) - Standardize signal logging to CSV include units - Provide live dashboards (matplotlib animation Plotly) for key traces - Event logs (impacts, saturations, limiter hits, FMU events, rollbacks)
UR-15	The framework should allow the user to compare simulation results across different configurations .	Partially	- Run multiple configs (e.g., PyFMI vs. FMPy, OpenSim vs. Modelica pendulum) and plot traces (q_ref, q_state, torques) side-by-side - FMU/OpenSim runs can be parameterized via Python and re-run to produce comparable outputs.	- Add a RunManager that: accepts a YAML sweep (grid/random search of parameters, solvers, stepsizes), assigns run IDs, and captures config - Standardize results to a common ResultStore with metadata (units, value refs, component)
UR-16	The framework should support result export and post-processing functionality.	Open	- Current notebooks/scripts already collect time series (q_ref, q_state, torques) and plot with matplotlib. - Raw data are available for	- Add a unified ResultStore - Include metadata: run ID, configs, tool versions (FMPy, OpenSim), units, value references, sampling rates

1. Configuration & Orchestration:

- Implement **YAML-based system configuration** (model locations, parameters, initial states, outputs)
- **Add automatic configuration generation** from FMUs, .osim files, FEM files
 - read metadata → create config templates
- **Build a Configuration Manager:**
 - validate connections, parameters, and unit compatibility
- **Extend the Simulation Master:**
 - Determine execution order (dependency graph)
 - Orchestrate FMUs, OpenSim, FEM in unified loop
 - Manage hybrid events (zero-crossings, reinitializations, discrete jumps).

2. Heterogeneous Modeling:

- **FMU integration:**
 - finalize wrapper (already working with FMPy)
- **OpenSim integration:**
 - implement generic wrapper (parameters, coordinates, actuators, motion export)
- **FEM integration:**
 - Define feasible toolchain (CalculiX wrapper vs. OpenCFS for non-contact problems)
 - Prototype simple FEM subsystem (rod elasticity or Hertz contact)
 - Test co-simulation exchange with SysSimX.

3. Hybrid Behavior & Events

- Extend **interface protocol with event handling** methods
- Demonstrate on demo models:
 - **Wall impact** (continuous & discrete)
 - **Hybrid pendulum with FEM contact**
- Implement test cases for rollback and communication step synchronization.

4. User Interaction & Visualization (planned later)

- Define result handling API:
 - consistent logging
 - metadata storage
- Implement result comparison across configs.
- Add post-processing
- Later: Visual representation of system models (graph layout)

1. Modelica Association Project "FMI". (2022, October). **Functional Mock-up Interface Specification**, Version 3.0. <https://fmi-standard.org/>
2. Sima, H. (2024). 317.047 **Computational Bionics: Modelling and Simulation** [Lecture slides]. Institute of Lightweight Design and Structural Biomechanics, TU Wien. <https://www.ilsb.tuwien.ac.at>
3. Urquía Moraleda, A., & Martín Villalba, C. (2018, April). **Modeling and simulation in engineering using Modelica**. Editorial UNED. <https://e-spacio.uned.es/entities/publication/cbfb24a9-2ec0-4afa-aa9c-243e1309da6e/full>
4. Wetter, M. (2019, February 15). **Functional Mock-up Interface** [Presentation]. Simulation Research Group, Lawrence Berkeley National Laboratory.
5. OpenModelica Project. (2025). **OpenModelica Python Interface** (OMPpython) — OpenModelica User's Guide. <https://openmodelica.org/doc/OpenModelicaUsersGuide/latest/ompython.html>
6. Dassault Systèmes (2025). **FMPy — Python package for working with Functional Mock-up Units (FMUs)** [GitHub repository]. <https://github.com/CATIA-Systems/FMPy>
7. OpenSim. (2025). **Scripting in Python — OpenSim Documentation**. <https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53085346/Scripting+in+Python>